

GETTING STARTED GUIDE

AURA CODE

A model-agnostic autonomous coding agent. Natural-language task → read codebase → plan → execute → verify → report.

v0.10.0

`npm install -g aura-code`

TypeScript

MIT

01 Before You Start — Install Node.js

PREREQUISITE

Aura Code runs on Node.js. You need Node.js installed before anything else works — `npm` (Node's package manager) comes bundled with it.

What you need

- **Node.js 20 or newer** (Node 22 is tested and recommended)
- A terminal — Terminal.app on macOS, any shell on Linux, PowerShell or WSL on Windows
- An internet connection for installing packages and calling AI providers

macOS

- 1 Install **Homebrew** if you don't have it, then install Node:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
brew install node
```

Linux (Ubuntu / Debian)

- 1 Use NodeSource's setup script for a recent version — the Ubuntu repo default is usually too old:

```
# Node 22 LTS
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt-get install -y nodejs
```

Windows

- 1 Download the LTS installer from **nodejs.org** and run it, or via winget:

```
winget install OpenJS.NodeJS.LTS
```

Aura's terminal UI works best inside **WSL (Windows Subsystem for Linux)** or Windows Terminal — plain `cmd.exe` has limited support for the interactive modal interface.

Verify the install

```
node --version # should print v20.x or higher
npm --version
```

WHY THIS MATTERS

Everything after this step — installing Aura itself, running it, connecting Telegram — depends on a working `node` and `npm`. If either command is "not found," stop here and fix that first.

Aura is published on npm and installs as two global commands: `aura` and `dic`.

1 Install it globally:

```
npm install -g aura-code
```

2 Confirm both commands resolve:

```
aura --version  
which aura  
which dic
```

IF `AURA` OR `DIC` EVER GO MISSING

Global npm links can be lost — for example if something runs an untargeted `npm unlink` or `npm uninstall -g` from the wrong directory. Your build isn't gone; the global symlink is. Re-link it from the project directory:

```
cd path/to/aura-code  
npm run build  
npm link
```

If you don't have the source locally, just reinstall: `npm install -g aura-code`. Run `aura --doctor --fix` afterward to catch anything else out of place.

Keeping it up to date

```
npm update -g aura-code
```

Aura also self-checks: `aura --doctor` scans its own build, config, dependencies, environment, and git state for problems; add `--fix` to let it attempt repairs, or `--offline` to skip the network version check.

Aura needs at least one AI provider API key before it can do anything. The easiest path is the built-in setup wizard.

- 1 From any directory, run Aura with no arguments — this opens the interactive setup wizard on first use:

```
aura
```

- 2 The wizard walks you through picking a provider and model, and asks for your API key. (See Section 04 for how to get your first keys.) Once configured, you land in Aura's modal terminal UI.

- 3 You can also skip straight to a task, if a key is already set as an environment variable:

```
# One-shot task, no interactive session
aura "add a logout button to the navbar"

# Fully autonomous – no confirmation prompts
aura --auto "fix the failing tests"

# Force a specific model or provider
aura -m claude-opus-4-5-20251001 "refactor the auth module"
aura --provider openrouter -m gpt-4o "add unit tests"

# No API key at all – fully local via Ollama
aura --profile local -m ollama/llama3.2 "explain this codebase"
```

| Inside the TUI

Aura's terminal UI is modal, like vim: **INSERT** mode for typing tasks, **SCROLL** mode (press `Esc`) for paging back through the session with `j`/`k`. Press `i` or `Enter` to return to typing. Type `:help` at any time for the full command list (Section 06 has it printed in full).

Aura is model-agnostic — it works with any provider. Here are three good starting points: two low-friction gateways and one direct provider, so you have real model choice from day one.

1 OpenCode Go

opencode.ai/docs/go

A low-cost subscription giving access to a curated set of open coding models (DeepSeek, GLM, Kimi K2, and others), hosted for stable global access — a good option from Vietnam specifically, since latency and availability to some providers can be inconsistent.

- Sign in at **opencode.ai** and subscribe to **OpenCode Go** (via OpenCode Zen)
- Copy the API key it issues you
- In Aura, this is added as a custom OpenAI-compatible provider — either through the `:provider` wizard, or by hand in `.aura.json` (see the config block on the next page)

2 OpenRouter

openrouter.ai/keys

A single gateway to hundreds of models from OpenAI, Anthropic, Google, Meta, and more — useful for trying different models without collecting a separate key for each provider.

- Sign up at **openrouter.ai** (email, Google, or GitHub)
- Add credit (prepaid — a few dollars is enough to start)
- Go to **openrouter.ai/keys**, click **Create Key**, and copy it immediately — it's only shown once
- Set it as `OPENROUTER_API_KEY` (see table below)

3 Anthropic (Claude)

console.anthropic.com

Direct access to Claude models (Opus, Sonnet, Haiku) — Aura's own default/reference provider, worth having a direct key for even if you mostly route through a gateway.

- Sign up or log in at **console.anthropic.com**
- Go to **API Keys**, create a new key, and copy it
- Set it as `ANTHROPIC_API_KEY`

Option A — the wizard (easiest)

Inside Aura, run the provider setup wizard — it prompts you for the provider, model, and key, and stores it for you:

```
:provider
```

To set just a key for the provider you're already on:

```
:apikey <your-key>
```

Option B — environment variables

Add these to your shell profile (`~/.bashrc` , `~/.zshrc`) so they're available in every session:

```
# ~/.bashrc or ~/.zshrc
export ANTHROPIC_API_KEY=sk-ant-...
export OPENROUTER_API_KEY=sk-or-...
```

Then reload: `source ~/.bashrc`

ENV VAR	PROVIDER
ANTHROPIC_API_KEY	Claude models
OPENAI_API_KEY	GPT models
GOOGLE_API_KEY	Gemini models
XAI_API_KEY	Grok models
OPENROUTER_API_KEY	OpenRouter (all models)
XIAOMI_API_KEY	Xiaomi MiMo
ZHIPU_API_KEY	Zhipu GLM (Z.ai)

Option C — project config (.aura.json)

For per-project settings, or to add a custom OpenAI-compatible provider like OpenCode Go, drop an `.aura.json` in your project root:

```
{
  "model": "claude-sonnet-4-5-20251001",
  "mode": "auto",
  "providers": [
    {
      "name": "OpenCodeGo",
      "baseUrl": "https://opencode.ai/zen/v1",
      "apiKeyEnv": "OPENCODE_GO_API_KEY",
      "prefixes": ["opencode-go/"],
      "models": [
        { "id": "opencode-go/kimi-k2.7-code", "name": "Kimi K2.7 Code", "speed":
"Fast" }
      ]
    }
  ],
  "fallbacks": ["gpt-4o-mini", "gemini-2.5-flash"]
}
```

NOTE

Custom providers (like OpenCode Go) are configured as OpenAI-compatible endpoints. Confirm the current base URL and model IDs in OpenCode's own docs before relying on the example above — provider endpoints do change.

CLI flags always override `.aura.json` , which always overrides environment variables.

Aura's Telegram bot lets you send it tasks, get voice replies, and control your PC remotely from your phone.

1 In Telegram, message **@BotFather** and send `/newbot`. Follow the prompts to name your bot; BotFather gives you a bot token — a long string like `123456:ABC-DEF...`.

2 In Aura, just ask for it in plain language — for example:

```
connect telegram
```

Aura will ask you back for the bot token. Paste it in when prompted.

3 Once connected, message your bot from your phone. Useful bot commands once it's running: `/status`, `/stop`, `/approve-all`, `/safety_off`, `/ls`, `/read`, `/run`, `/search`.

TREAT THE BOT TOKEN LIKE A PASSWORD

Anyone with your bot token can send commands to your PC through the bot. Never post it in a public chat, repo, or issue tracker. If it ever leaks, revoke it immediately in @BotFather (`/revoke`) and generate a new one.

06 Connecting Gmail, Google Drive & Others

INTEGRATIONS

The same conversational pattern applies to other services — you ask, Aura asks back for whatever credential it needs.

- 1 Just tell Aura what you want to connect, in plain language:

```
connect gmail
connect google drive
```

Aura will prompt you for the credential or walk you through whatever authentication step that service needs — this may mean pasting a token, or it may open a browser for an OAuth sign-in, depending on the service.

BE PRECISE ABOUT WHICH SERVICE

"Connect Gmail" and "connect Google Drive" are two separate connections with two separate credentials, even though both are Google products — don't assume connecting one connects the other.

IF A CONNECTION DOESN'T BEHAVE AS EXPECTED

Integration surfaces evolve fastest of anything in Aura. If a service doesn't connect the way you expect, run `:help` to confirm what's currently wired up, or `aura --doctor` to check your environment and config for missing pieces. Don't guess at a flag or command that isn't in `:help` — ask Aura directly instead ("how do I connect X") and let it tell you what it actually supports right now.

07 The Powerful Stuff — Complex Commands

WORKFLOWS

Once the basics feel comfortable, these are the commands that make Aura genuinely different from a plain chat assistant.

| :machina — self-verifying execution

```
:machina refactor the payment module to use the new API client
```

Runs the task, then verifies its own result and automatically retries if verification fails — best for anything you don't want to babysit.

| :council — parallel specialist review

```
:council should we migrate this service to gRPC?
```

Spins up 2-3 parallel read-only specialist agents to look at the question from different angles, then synthesizes their findings into one answer.

| :ecclesia — deep independent research

```
:ecclesia evaluate our current rate-limiting strategy
:ecclesia compare Postgres vs. SQLite for our use case --panel gpt-4o --seats 5
```

Five independent research agents investigate separately, then a verdict is synthesized and saved to `council/*.md` and `council/*.html`.

| :workflow — multi-step, named pipelines

```
:workflow release "run tests" "bump version" "update changelog" "push and tag"
```

Runs each step in order as its own task. If a step fails, resume later without repeating the steps that already succeeded:

```
:resume-workflow <id>
```

| --orchestrate — multi-agent mode

```
aura --orchestrate "add multi-tenant support to the API layer"
```

Splits the task across a Researcher → Coder → Reviewer pipeline instead of one agent doing everything sequentially.

| :dream / :rem / :mine — memory tools

```
:dream          # consolidate recent episodes into a dream entry
:rem            # show reconciled memory / latest dream
:mine --refine   # cluster episodes into training-data/*.jsonl
```

These let Aura build durable memory of past sessions rather than starting fresh each time.

08

Command Reference — Outside Aura

TERMINAL /
SHELL

Run these from your regular shell, before or without starting the TUI.

– Install & version –

COMMAND	WHAT IT DOES
<code>npm install -g aura-code</code>	Install Aura globally
<code>npm update -g aura-code</code>	Update to the latest version
<code>aura --version</code>	Print installed version

– Running a task –

COMMAND	WHAT IT DOES
<code>aura</code>	Open interactive TUI / first-run wizard
<code>aura "task"</code>	One-shot task, no interactive session
<code>aura --auto "task"</code>	Fully autonomous — no confirmation prompts
<code>aura --readonly "task"</code>	Read-only analysis, no edits
<code>aura --orchestrate "task"</code>	Multi-agent: Researcher → Coder → Reviewer
<code>aura --verify "task"</code>	Post-task checks with auto-retry
<code>aura -m <id> "task"</code>	Force a specific model
<code>aura --provider <name> "task"</code>	Force a specific provider
<code>aura --profile local -m ollama/<model></code>	Run fully local via Ollama, no API key

– Workflows & diagnostics –

COMMAND	WHAT IT DOES
<code>aura --workflow <name> "s1" "s2"</code>	Create and run a named multi-step workflow
<code>aura --resume-workflow <id></code>	Resume a paused/failed workflow
<code>aura --workflows</code>	List all saved workflows
<code>aura --doctor</code>	Scan Aura's build, config, deps, env, git
<code>aura --doctor --fix</code>	Scan and attempt automatic repairs
<code>aura --doctor --offline</code>	Same, skip the network version check

– Resilience flags –

FLAG	WHAT IT DOES
<code>--rate-limit-rpm <n></code>	Cap requests per minute (0 = unlimited)
<code>--rate-limit-tpm <n></code>	Cap tokens per minute (Google only)
<code>--max-retries <n></code>	Max retry attempts on 429 / 5xx (default 5)
<code>--fallback <model></code>	Fallback model if primary exhausts retries (repeatable)

09 Command Reference — Inside Aura

TUI COLON
COMMANDS

Type these once you're inside the Aura TUI. This is the authoritative `:help` list — type `:help` any time to see it live.

— Session —

<code>:id</code>	Show current chat ID
<code>:sessions</code>	List all saved sessions
<code>:resume [id]</code>	Resume latest or a specific session
<code>:new</code>	Start a fresh session
<code>:history</code>	Show turn count
<code>:clear-history</code>	Wipe history, keep session ID
<code>:save [title]</code>	Rename / save session
<code>:delete <id></code>	Delete a saved session

— Model / API —

<code>:model</code>	Interactive model selector
<code>:model <id></code>	Switch to a specific model
<code>:models</code>	List available models
<code>:provider</code>	Provider setup wizard
<code>:apikey <key></code>	Set key for current provider

— Workflows —

<code>:workflows</code>	List saved workflows
<code>:workflow <name> "s1"...</code>	Create & run a workflow
<code>:resume-workflow <id></code>	Resume paused/failed workflow
<code>:q add <prompt></code>	Enqueue a task
<code>:q list</code>	List queued tasks
<code>:q run <n></code>	Run queued task #n
<code>:q drop <n></code>	Remove queued task #n
<code>:q clear</code>	Wipe the queue
<code>:machina <task></code>	Self-verifying execution
<code>:council <task></code>	2-3 parallel specialists
<code>:ecclesia <topic></code>	5 independent researchers

— Memory / Side —

<code>:dream [full]</code>	Consolidate episodes into memory
<code>:rem</code>	Show reconciled memory / latest dream
<code>:mine [--refine]</code>	Mine episodes for patterns
<code>:research <topic></code>	Multi-step research pass
<code>:confess</code>	Auto-detect & confess an anomaly
<code>:confessions</code>	List all confessions
<code>:btw <question></code>	Quick side question, no history impact

— Voice —

<code>:speak</code>	Toggle reading replies aloud
---------------------	------------------------------

— Safety —

<code>:approve</code>	Toggle auto-approve
<code>:approve all</code>	Approve everything this session
<code>:approve off</code>	Re-enable confirmations

— Context / Stats —

<code>:context</code>	Show loaded project context
<code>:graph [refresh]</code>	Codebase knowledge graph
<code>:plans</code>	List saved execution plans
<code>:viz, :dashboard</code>	Open the memory dashboard
<code>:doctor [--fix]</code>	Scan Aura itself for issues
<code>/stats, /usage</code>	Token + cost usage this session
<code>/context</code>	Context health dashboard
<code>/clear, /reset</code>	Reset usage stats

— General —

<code>:quit, :q, /exit</code>	Exit Aura
<code>Esc</code>	Enter SCROLL mode
<code>i / Enter</code>	Return to INSERT mode
<code>j / k</code>	Scroll down / up (SCROLL mode)

10 You're Set Up

WRAP - UP

You now have Node.js, Aura Code itself, three provider keys, and Telegram wired up — everything needed to run real, autonomous tasks against your own codebase.

Quick recap

- Install Node.js 20+, then `npm install -g aura-code`
- Run `aura` and complete the setup wizard, or set keys via env vars
- Get keys from OpenCode Go, OpenRouter, and Anthropic (Section 04)
- Ask Aura in plain language to connect Telegram, Gmail, Drive, or anything else it supports
- When you're ready to go further, reach for `:machina`, `:council`, `:ecclesia`, and `:workflow`

IF SOMETHING EVER BREAKS

Run `aura --doctor --fix` first. If `aura` or `dic` stop resolving entirely, it's almost always a lost global npm link, not a broken build — re-link from source with `npm link`, or reinstall with `npm install -g aura-code`.

github.com/milodule3-debug/aura-code · MIT Licensed · Built by AI agents, orchestrated by humans.